184 hw 3

Jack

2026-11-09

Package

```
library(ggplot2)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(tidyr)
library(readr)
library(sf)
```

```
## Warning: package 'sf' was built under R version 4.5.3
```

```
## Linking to GEOS 3.14.1, GDAL 3.12.1, PROJ 9.7.1; sf_use_s2() is TRUE
```

```
library(jsonlite)
library(httr)

vehicles <- ggplot2::mpg
gems <- ggplot2::diamonds
```

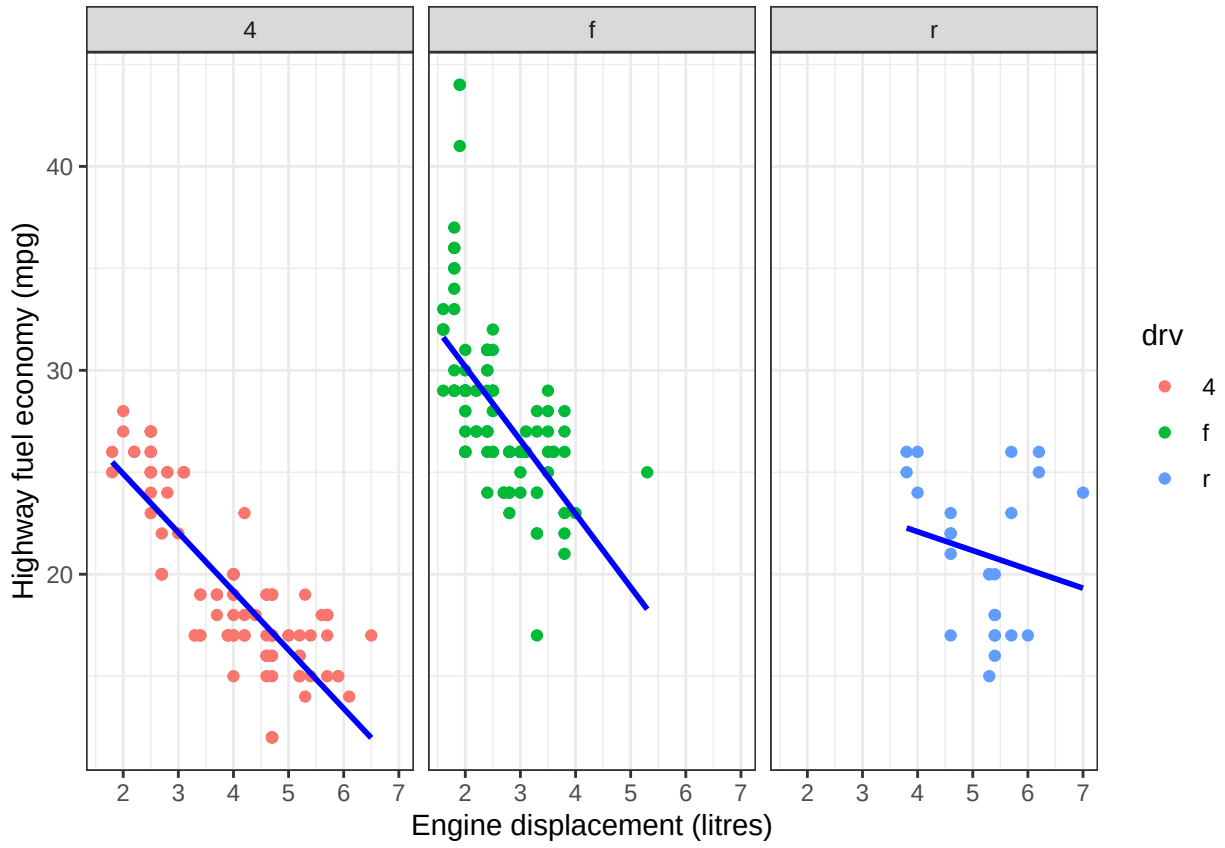
Problem 1: Reproduce and explain plots

Part A

```
p1a <- ggplot(vehicles, aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv)) +
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
  facet_wrap(~ drv) +
  labs(
    x = "Engine displacement (litres)",
    y = "Highway fuel economy (mpg)",
    color = "drv"
  ) +
  theme_bw()
```

```
p1a
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



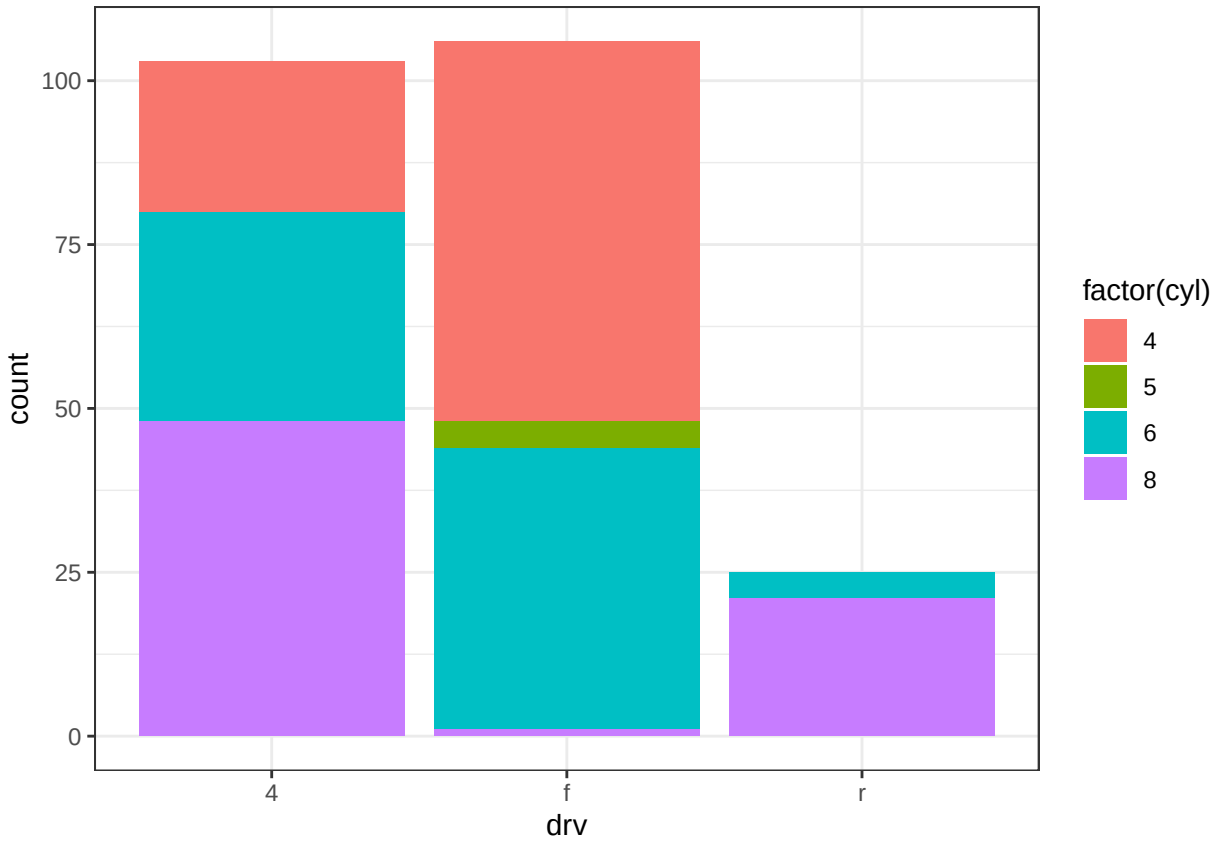
Explanation:

facet_wrap(~drv) splits the data into three panels, one per drv value. geom_smooth() fits its line separately inside each panel, using only the rows shown there. So each panel gets its own line even though color is mapped only inside geom_point. The facet instead of the color aesthetic, is what creates the three separate fits.

Part B

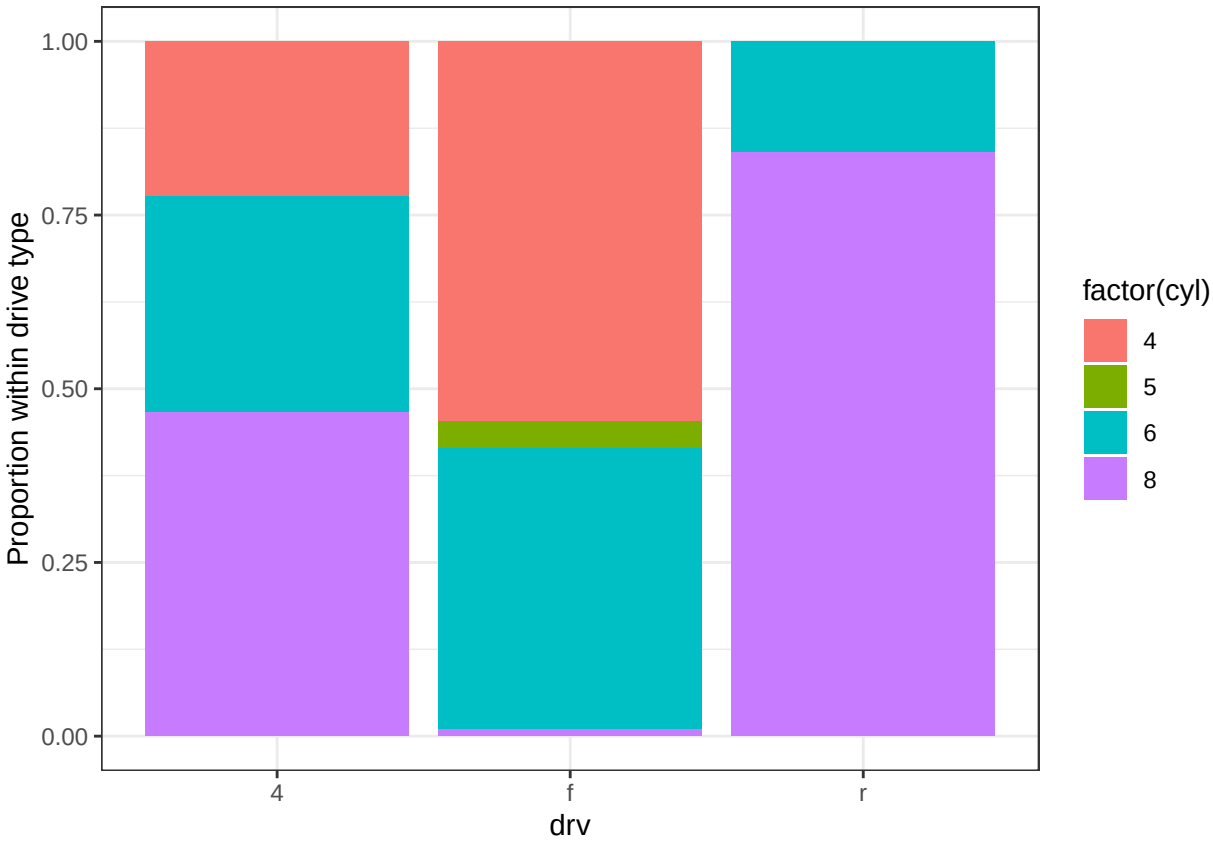
```
p1b_count <- ggplot(vehicles, aes(x = drv, fill = factor(cyl))) +
  geom_bar() +
  labs(x = "drv", y = "count", fill = "factor(cyl)") +
  theme_bw()

p1b_count
```




```
p1b_prop <- ggplot(vehicles, aes(x = drv, fill = factor(cyl))) +
  geom_bar(position = "fill") +
  labs(x = "drv", y = "Proportion within drive type", fill = "factor(cyl)") +
  theme_bw()

p1b_prop
```



Explanation:

position = "fill" rescales every bar to the same height of 1. So each bar shows proportions, not raw counts. Drive types with few cars get stretched to the same height as drive types with a lot of cars. Equal bar heights only represents cylinder proportions of each group that equates to 100 percents. The left plot is what shows the real sample sizes.

Part C

```
gems_small <- gems %>% filter(carat <= 3)

n_total    <- nrow(gems)
n_retained <- nrow(gems_small)
n_excluded <- n_total - n_retained

cat("Records retained:", n_retained, "\n")
```

```
## Records retained: 53908
```

```
cat("Records excluded:", n_excluded, "\n")
```

```
## Records excluded: 32
```

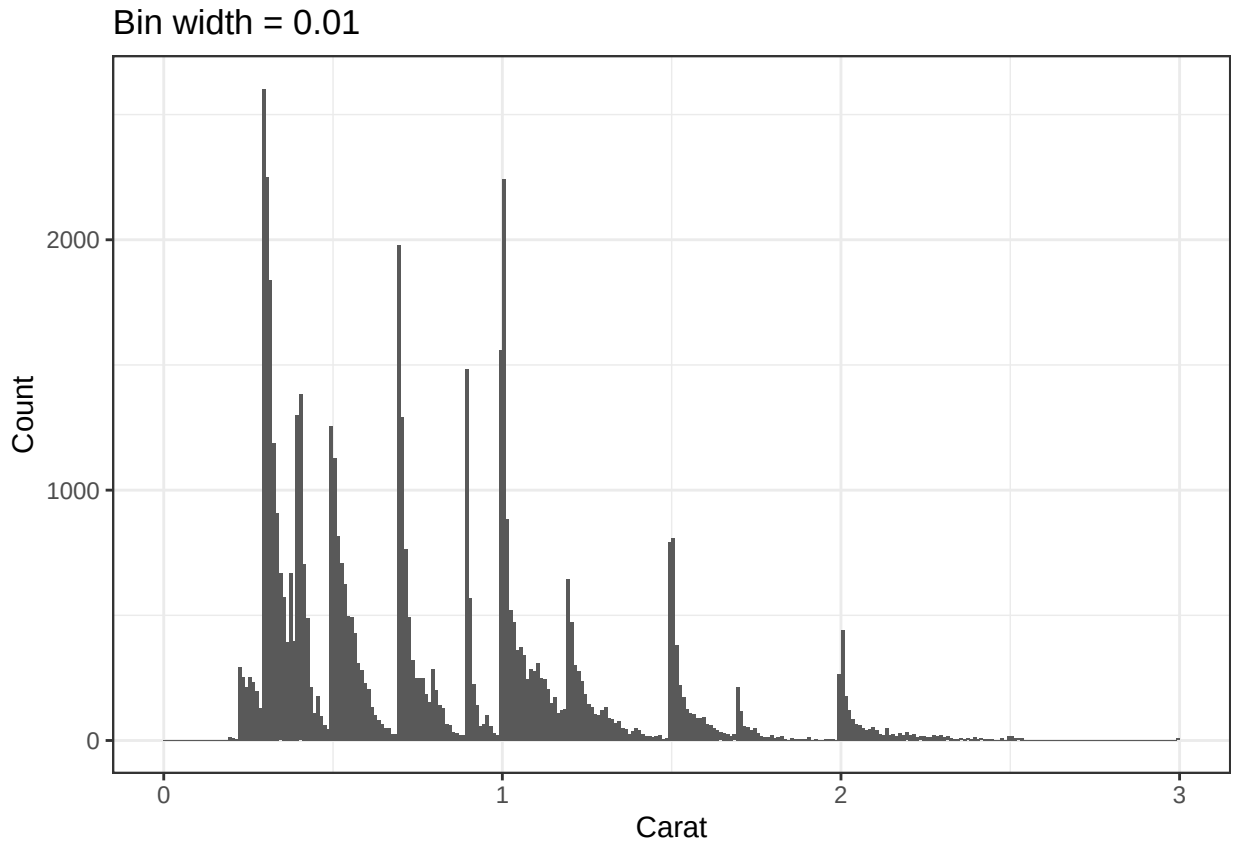
```
x_range <- c(0, 3)

p_fine <- ggplot(gems_small, aes(x = carat)) +
```

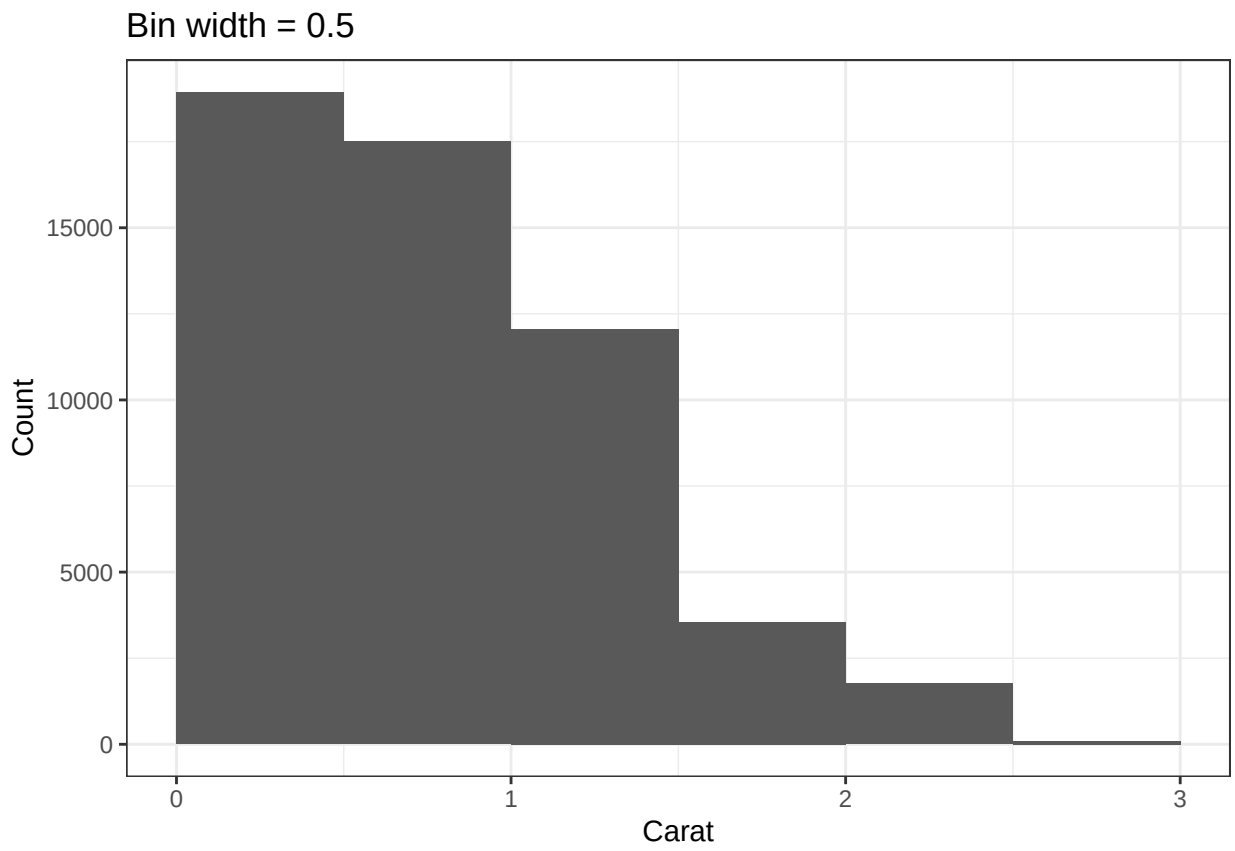
```
  geom_histogram(binwidth = 0.01, boundary = 0, closed = "right") +
  scale_x_continuous(limits = x_range) +
  labs(title = "Bin width = 0.01", x = "Carat", y = "Count") +
  theme_bw()

p_coarse <- ggplot(gems_small, aes(x = carat)) +
  geom_histogram(binwidth = 0.5, boundary = 0, closed = "right") +
  scale_x_continuous(limits = x_range) +
  labs(title = "Bin width = 0.5", x = "Carat", y = "Count") +
  theme_bw()

p_fine
```




```
p_coarse
```



Diamonds

```
carat_counts <- gems %>%
  filter(carat %in% c(0.99, 1.00)) %>%
  count(carat)

print(carat_counts)
```

```
## # A tibble: 2 x 2
##   carat     n
##   <dbl> <int>
## 1  0.99    23
## 2  1     1558
```

Feature easier to see with narrow bins:

The fine bins show a sharp dip just below 1.00 carat and a tall spike right at 1.00 carat. Cutters shave stones down to just under a round number when it isn't worth the cost of keeping a few extra points. It then rounds up when a cheap cut gets them over the line whicg results in stones piling up near whole and half carats. The 0.5-width bins merge that whole region into a single wide bar and hide the pattern completely.

PROBLEMS 2-3

```r
dir.create("raw_data", showWarnings = FALSE)
```

```r
get_all_rows <- function(resource_url, page_size = 1000) {
  all_rows <- list()
  offset <- 0
  repeat {
    url <- paste0(resource_url, "?$limit=", page_size, "&$offset=", offset)
    page <- fromJSON(url, flatten = TRUE)
    if (length(page) == 0 || nrow(page) == 0) break
    all_rows[[length(all_rows) + 1]] <- page
    offset <- offset + page_size
    if (nrow(page) < page_size) break
  }
  bind_rows(all_rows)
}

libraries_raw <- get_all_rows("https://data.cityofchicago.org/resource/x8fc-8rcq.json")
write_csv(libraries_raw, "raw_data/chicago_libraries_raw.csv")
```

```r
areas_raw <- st_read(
  "https://data.cityofchicago.org/resource/igwz-8jzy.geojson",
  quiet = TRUE
)
st_write(areas_raw, "raw_data/chicago_community_areas_raw.geojson", delete_dsn = TRUE)
```

```
## Deleting source `raw_data/chicago_community_areas_raw.geojson' failed
## Writing layer `chicago_community_areas_raw' to data source
##   `raw_data/chicago_community_areas_raw.geojson' using driver `GeoJSON'
## Writing 77 features with 5 fields and geometry type Multi Polygon.
```

```r
data_sources <- tibble(
  dataset = c("Chicago Public Libraries - Locations", "Chicago Community Areas"),
  source = c(
    "City of Chicago Data Portal (data.cityofchicago.org), resource x8fc-8rcq",
    "City of Chicago Data Portal (data.cityofchicago.org), resource igwz-8jzy"
  ),
  url = c(
    "https://data.cityofchicago.org/resource/x8fc-8rcq.json",
    "https://data.cityofchicago.org/resource/igwz-8jzy.geojson"
  ),
  accessed_date = as.character(Sys.Date())
)

write_csv(data_sources, "data_sources.csv")
```

PROBLEM 2:

```r
n_before_cleaning <- nrow(libraries_raw)

get_col <- function(df, nm) if (nm %in% names(df)) df[[nm]] else NA

lat_candidates <- c("location.latitude", "latitude", "location.lat")
lon_candidates <- c("location.longitude", "longitude", "location.lon", "location.long")
```

```r
pick_first_present <- function(df, candidates) {
  for (nm in candidates) if (nm %in% names(df)) return(df[[nm]])
  rep(NA_character_, nrow(df))
}

raw_lat <- pick_first_present(libraries_raw, lat_candidates)
raw_lon <- pick_first_present(libraries_raw, lon_candidates)

if (all(is.na(raw_lat)) && "location.coordinates" %in% names(libraries_raw)) {
  coords <- libraries_raw[["location.coordinates"]]
  raw_lon <- sapply(coords, function(x) if (length(x) == 2) x[1] else NA)
  raw_lat <- sapply(coords, function(x) if (length(x) == 2) x[2] else NA)
}

name_col <- if ("branch_" %in% names(libraries_raw)) {
  libraries_raw[["branch_"]]
} else if ("name" %in% names(libraries_raw)) {
  libraries_raw[["name"]]
} else {
  paste("Library", seq_len(nrow(libraries_raw)))
}

libraries_clean <- libraries_raw %>%
  mutate(
    latitude  = as.numeric(raw_lat),
    longitude = as.numeric(raw_lon),
    facility_name = name_col
  ) %>%
  filter(!is.na(latitude), !is.na(longitude)) %>%
  mutate(site_id = row_number()) %>%
  select(site_id, facility_name, latitude, longitude)

n_after_cleaning <- nrow(libraries_clean)
n_dropped_missing_coords <- n_before_cleaning - n_after_cleaning
n_duplicate_coords <- libraries_clean %>%
  count(latitude, longitude) %>%
  filter(n > 1) %>%
  nrow()

cat("Libraries before cleaning:", n_before_cleaning, "\n")
```

```
## Libraries before cleaning: 82
```

```r
cat("Libraries after cleaning:", n_after_cleaning, "\n")
```

```
## Libraries after cleaning: 82
```

```r
cat("Dropped for missing coordinates:", n_dropped_missing_coords, "\n")
```

```
## Dropped for missing coordinates: 0
```

```r
cat("Coordinate pairs shared by more than one record:", n_duplicate_coords, "\n")
```

```
## Coordinate pairs shared by more than one record: 0
```

```
sites_wgs84 <- st_as_sf(
  libraries_clean,
  coords = c("longitude", "latitude"),
  crs = 4326
)

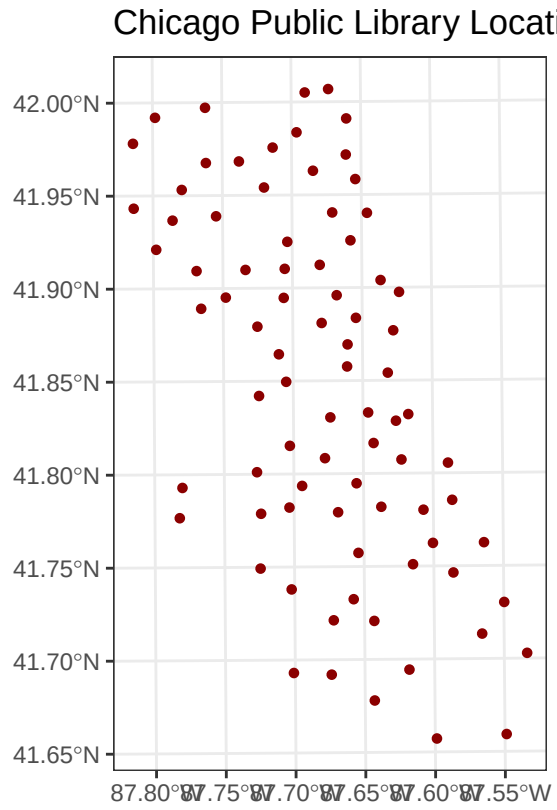
sites_m <- st_transform(sites_wgs84, crs = 26971)
cat("sites_m CRS:", st_crs(sites_m)$Name, "- units: meters\n")
```

```
## sites_m CRS: NAD83 / Illinois East - units: meters
```

p_sites

```
p_sites <- ggplot() +
  geom_sf(data = sites_m, color = "darkred", size = 1.2) +
  labs(
    title = "Chicago Public Library Locations",
    caption = "Source: City of Chicago Data Portal (x8fc-8rcq), accessed 2026"
  ) +
  theme_bw()

p_sites
```



buffers and union areas

```
buffer_500  <- st_buffer(sites_m, dist = 500)
buffer_1000 <- st_buffer(sites_m, dist = 1000)
```

```r
union_500  <- st_union(buffer_500)
union_1000 <- st_union(buffer_1000)

area_500_km2  <- as.numeric(st_area(union_500))  / 1e6
area_1000_km2 <- as.numeric(st_area(union_1000)) / 1e6

stopifnot(area_1000_km2 >= area_500_km2)

buffer_summary <- tibble(
  radius_m = c(500, 1000),
  union_area_km2 = c(area_500_km2, area_1000_km2)
)

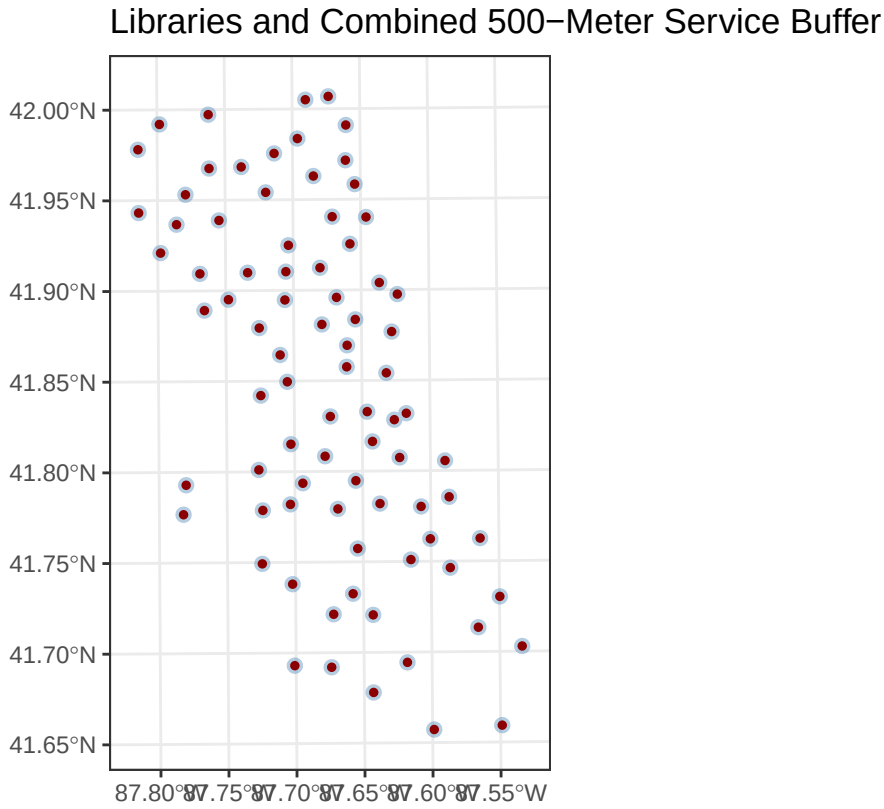
print(buffer_summary)
```

```
## # A tibble: 2 x 2
##   radius_m union_area_km2
##      <dbl>          <dbl>
## 1      500           64.3
## 2     1000          250.
```

p_buffers

```r
p_buffers <- ggplot() +
  geom_sf(data = union_500, fill = "steelblue", alpha = 0.4, color = NA) +
  geom_sf(data = sites_m, color = "darkred", size = 1) +
  labs(
    title = "Libraries and Combined 500-Meter Service Buffer",
    caption = "Source: City of Chicago Data Portal (x8fc-8rcq), accessed 2026"
  ) +
  theme_bw()

p_buffers
```



Source: City of Chicago Data Portal (x8fc−8rcq), accessed 2026

PROBLEM 3:

```
areas_clean <- areas_raw %>%
  st_make_valid() %>%
  mutate(
    district_id = as.character(area_numbe),
    district_name = community
  ) %>%
  filter(!is.na(district_id), !st_is_empty(geometry)) %>%
  select(district_id, district_name, geometry)

n_areas_before <- nrow(areas_raw)
n_areas_after  <- nrow(areas_clean)
n_dup_ids <- areas_clean %>% st_drop_geometry() %>%
  count(district_id) %>% filter(n > 1) %>% nrow()

cat("Community areas before cleaning:", n_areas_before, "\n")
```

```
## Community areas before cleaning: 77
```

```
cat("Community areas after cleaning:", n_areas_after, "\n")
```

```
## Community areas after cleaning: 77
```

```
cat("Duplicate district IDs:", n_dup_ids, "\n")
```

```
## Duplicate district IDs: 0
```

```r
areas_m <- st_transform(areas_clean, crs = st_crs(sites_m))
```

match sites to exactly one area

```r
matches <- st_intersects(sites_m, areas_m)
n_matches_per_site <- lengths(matches)

site_match_audit <- tibble(
  site_id = sites_m$site_id,
  n_matches = n_matches_per_site
) %>%
  mutate(
    matched_district_id = ifelse(
      n_matches == 1,
      areas_m$district_id[sapply(matches, function(x) if (length(x) == 1) x else NA)],
      NA_character_
    )
  )

write_csv(site_match_audit, "site_match_audit.csv")

n_zero_matches <- sum(n_matches_per_site == 0)
n_one_match    <- sum(n_matches_per_site == 1)
n_multi_match  <- sum(n_matches_per_site > 1)

cat("Sites matching zero areas:", n_zero_matches, "\n")
```

```
## Sites matching zero areas: 0
```

```r
cat("Sites matching exactly one area:", n_one_match, "\n")
```

```
## Sites matching exactly one area: 82
```

```r
cat("Sites matching more than one area:", n_multi_match, "\n")
```

```
## Sites matching more than one area: 0
```

district_counts

```r
site_counts <- site_match_audit %>%
  filter(n_matches == 1) %>%
  count(matched_district_id, name = "n_sites") %>%
  rename(district_id = matched_district_id)

areas_m <- areas_m %>%
  mutate(area_km2 = as.numeric(st_area(geometry)) / 1e6)

district_counts <- areas_m %>%
  st_drop_geometry() %>%
  left_join(site_counts, by = "district_id") %>%
  mutate(
    n_sites = ifelse(is.na(n_sites), 0, n_sites),
    sites_per_km2 = n_sites / area_km2
  ) %>%
```

```r
  select(district_id, district_name, n_sites, area_km2, sites_per_km2)

stopifnot(all(district_counts$area_km2 > 0))
stopifnot(sum(district_counts$n_sites) == n_one_match)

print(district_counts)
```

```
##    district_id          district_name n_sites  area_km2 sites_per_km2
## 1            1            ROGERS PARK       1  4.762222    0.20998603
## 2            2             WEST RIDGE       2  9.144398    0.21871314
## 3            3                 UPTOWN       2  6.047606    0.33070937
## 4            4          LINCOLN SQUARE       1  6.628875    0.15085516
## 5            5            NORTH CENTER       0  5.300528    0.00000000
## 6            6               LAKE VIEW       2  8.102552    0.24683581
## 7            7            LINCOLN PARK       1  8.204898    0.12187843
## 8            8         NEAR NORTH SIDE       2  7.123455    0.28076265
## 9            9             EDISON PARK       0  2.939122    0.00000000
## 10          10            NORWOOD PARK       2 11.330411    0.17651610
## 11          11          JEFFERSON PARK       0  6.026474    0.00000000
## 12          12             FOREST GLEN       1  8.280562    0.12076475
## 13          13              NORTH PARK       1  6.530059    0.15313797
## 14          14             ALBANY PARK       1  4.974258    0.20103502
## 15          15            PORTAGE PARK       3 10.237592    0.29303767
## 16          16             IRVING PARK       1  8.325203    0.12011719
## 17          17                 DUNNING       1  9.624328    0.10390336
## 18          18              MONTCLARE       1  2.561942    0.39032890
## 19          19          BELMONT CRAGIN       1 10.135702    0.09866115
## 20          20                 HERMOSA       0  3.028842    0.00000000
## 21          21                AVONDALE       0  5.136686    0.00000000
## 22          22            LOGAN SQUARE       1  9.295688    0.10757676
## 23          23           HUMBOLDT PARK       3  9.335017    0.32137060
## 24          24               WEST TOWN       2 11.851029    0.16876171
## 25          25                  AUSTIN       3 18.511398    0.16206231
## 26          26     WEST GARFIELD PARK       1  3.353148    0.29822719
## 27          27     EAST GARFIELD PARK       0  5.005937    0.00000000
## 28          28         NEAR WEST SIDE       3 14.724487    0.20374224
## 29          29          NORTH LAWNDALE       1  8.313687    0.12028358
## 30          30          SOUTH LAWNDALE       2 11.891480    0.16818764
## 31          31        LOWER WEST SIDE       1  7.576346    0.13198976
## 32          32                   LOOP       1  4.304729    0.23230268
## 33          33        NEAR SOUTH SIDE       0  4.623771    0.00000000
## 34          34           ARMOUR SQUARE       1  2.579571    0.38766132
## 35          35                 DOUGLAS       2  4.273987    0.46794716
## 36          36                 OAKLAND       0  1.571365    0.00000000
## 37          37              FULLER PARK       0  1.850330    0.00000000
## 38          38        GRAND BOULEVARD       1  4.505119    0.22196973
## 39          39                 KENWOOD       1  2.700862    0.37025214
## 40          40        WASHINGTON PARK       0  3.936678    0.00000000
## 41          41               HYDE PARK       0  4.190446    0.00000000
## 42          42                 WOODLAWN       2  5.371225    0.37235453
## 43          43             SOUTH SHORE       1  7.600679    0.13156719
## 44          44                 CHATHAM       1  7.647873    0.13075531
## 45          45             AVALON PARK       1  3.237937    0.30883858
## 46          46           SOUTH CHICAGO       1  8.665308    0.11540270
```

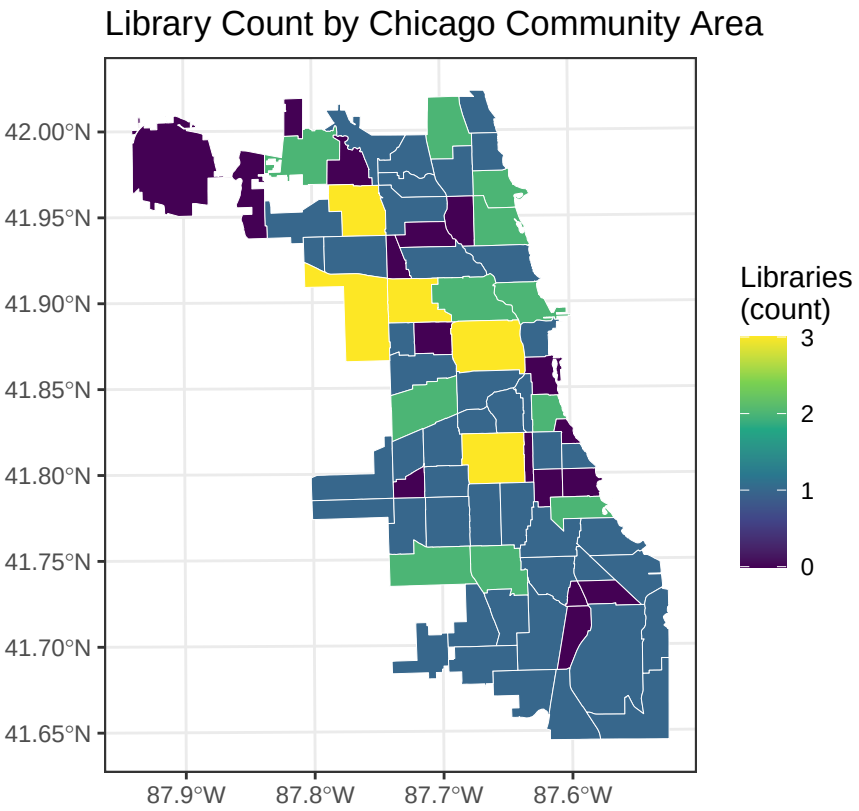
```
## 47          47               BURNSIDE       0  1.578985    0.00000000
## 48          48        CALUMET HEIGHTS       0  4.536124    0.00000000
## 49          49               ROSELAND       1 12.478200    0.08013977
## 50          50                PULLMAN       0  5.481445    0.00000000
## 51          51          SOUTH DEERING       1 28.223787    0.03543111
## 52          52              EAST SIDE       1  7.733437    0.12930862
## 53          53           WEST PULLMAN       1  9.231363    0.10832637
## 54          54              RIVERDALE       1  9.140716    0.10940062
## 55          55              HEGEWISCH       1 13.560715    0.07374243
## 56          56         GARFIELD RIDGE       1 10.952452    0.09130375
## 57          57         ARCHER HEIGHTS       1  5.195391    0.19247829
## 58          58          BRIGHTON PARK       1  7.050702    0.14182984
## 59          59          MCKINLEY PARK       1  3.663345    0.27297459
## 60          60             BRIDGEPORT       1  5.415485    0.18465568
## 61          61               NEW CITY       3 12.508232    0.23984205
## 62          62            WEST ELSDON       0  3.038971    0.00000000
## 63          63              GAGE PARK       1  5.693576    0.17563654
## 64          64               CLEARING       1  6.609637    0.15129425
## 65          65              WEST LAWN       1  7.647371    0.13076389
## 66          66           CHICAGO LAWN       1  9.130496    0.10952308
## 67          67         WEST ENGLEWOOD       1  8.170641    0.12238942
## 68          68              ENGLEWOOD       1  7.957390    0.12566935
## 69          69 GREATER GRAND CROSSING       1  9.183798    0.10888741
## 70          70                ASHBURN       2 12.584727    0.15892280
## 71          71         AUBURN GRESHAM       2  9.760930    0.20489851
## 72          72                BEVERLY       1  8.247901    0.12124296
## 73          73     WASHINGTON HEIGHTS       1  7.398434    0.13516375
## 74          74        MOUNT GREENWOOD       1  7.022042    0.14240871
## 75          75            MORGAN PARK       1  8.535716    0.11715478
## 76          76                  OHARE       0 34.544798    0.00000000
## 77          77              EDGEWATER       1  4.501169    0.22216452
```

p_count and p_density

```
areas_for_map <- areas_m %>%
  left_join(district_counts %>% select(district_id, n_sites, sites_per_km2),
            by = "district_id")

p_count <- ggplot(areas_for_map) +
  geom_sf(aes(fill = n_sites), color = "white", size = 0.1) +
  scale_fill_viridis_c(name = "Libraries\n(count)") +
  labs(
    title = "Library Count by Chicago Community Area",
    caption = paste0(
      "Source: City of Chicago Data Portal (x8fc-8rcq, igwz-8jzy), accessed 2026. ",
      n_zero_matches, " site(s) left unassigned."
    )
  ) +
  theme_bw()

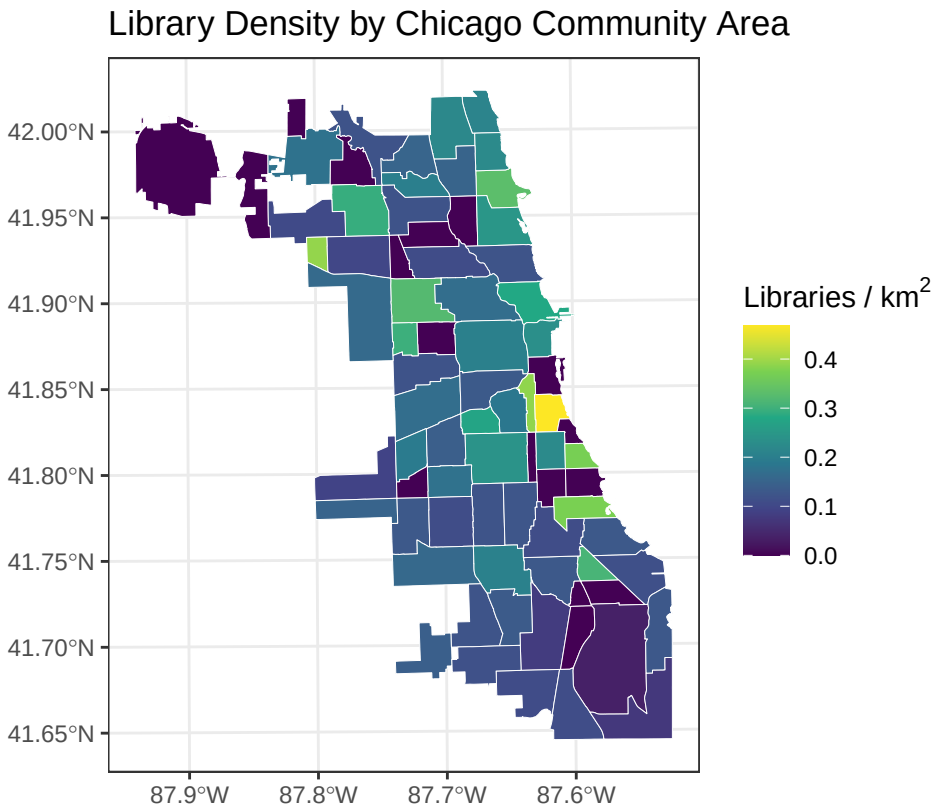
p_count
```



ıicago Data Portal (x8fc−8rcq, igwz−8jzy), accessed 2026. 0 site(s) left unassigned.

```
p_density <- ggplot(areas_for_map) +
  geom_sf(aes(fill = sites_per_km2), color = "white", size = 0.1) +
  scale_fill_viridis_c(name = expression("Libraries / " * km^2)) +
  labs(
    title = "Library Density by Chicago Community Area",
    caption = paste0(
      "Source: City of Chicago Data Portal (x8fc-8rcq, igwz-8jzy), accessed 2026. ",
      n_zero_matches, " site(s) left unassigned."
    )
  ) +
  theme_bw()

p_density
```



go Data Portal (x8fc−8rcq, igwz−8jzy), accessed 2026. 0 site(s) left unassigned.

Interpretation:

The raw count map tends to favor large community areas just because they cover more ground, not because libraries are more accessible there. The density map corrects for that by dividing by area. This way, small but well-served community areas can rank above large ones with only a few libraries spread over a lot of land. Comparing the two maps side by side shows which areas are genuinely well covered versus which just look that way because of their size.